13. Priority, nice, and resource limits (ulimit)

13. Priority, nice, and resource limits (ulimit)

Phase: Phase 4 — Processes and jobs
Estimated time: 1 day
Slug: `13-nice-priority`

Learning objectives

- Adjust niceness
- Inspect ulimit
- Explain why limits exist
- Connect to cgroups at a high level

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

A single heavy job can starve everything else on a machine, and a runaway process can exhaust memory or file descriptors and take a service down. `nice` and `renice` influence how the CPU scheduler shares time, while `ulimit` (resource limits) caps what a process may consume. Together they are basic tools for keeping shared systems predictable.

Core concepts

Niceness is a hint to the scheduler of how "nice" a process is to others. Values range from –20 (highest priority, least nice) to 19 (lowest priority, nicest); the default is 0. A higher niceness number therefore means lower priority: the process gets a smaller share of CPU when there is contention. Ordinary users can only increase niceness of their own processes; only root can lower it (use negative values). Niceness only matters when CPUs are busy; on an idle machine every process runs at full speed.

```
niceness:  -20 ........ 0 ........ 19
           greedy    default    polite
           high priority        low priority (higher number)

ps -o pid,ni,pri,cmd    NI = nice value, PRI = kernel priority
nice -n 10 cmd          start with niceness +10
renice -n 5 -p PID      change a running process
```

Resource limits (rlimits) are per process and inherited by children. Each has a *soft* limit (enforced) and a *hard* limit (ceiling for the soft one; only root can raise it).

`ulimit` flag	Resource
`-n`	Open file descriptors
`-u`	Number of user processes
`-v`	Virtual memory (KB)
`-s`	Stack size (KB)
`-c`	Core file size
`-t`	CPU time (seconds)
`-f`	Max file size created
`-a`	Show all
`-S` / `-H`	Select soft / hard limit

`ulimit` is a shell builtin and affects the current shell and its children. Persistent per-user limits live in `/etc/security/limits.conf` (applied by PAM); for services use `LimitNOFILE=` and friends in a systemd unit. Check a running process with `cat /proc/PID/limits`. For stronger control of CPU and memory in groups of processes, modern systems use cgroups (`systemd-run -p MemoryMax=...`).

Worked examples

See niceness:

```
ps -o pid,ni,cmd -p $$
# sample output:   PID  NI CMD
# sample output:  1201   0 -bash
nice
# sample output: 0
```

Start with lower priority:

```
nice -n 10 sh -c 'nice'
# sample output: 10
nice -n 19 gzip -9 bigfile &
```

Change a running process:

```
sleep 300 &
# sample output: [1] 6001
renice -n 5 -p 6001
# sample output: 6001 (process ID) old priority 0, new priority 5
ps -o pid,ni,cmd -p 6001
# sample output:   PID  NI CMD
# sample output:  6001   5 sleep 300
renice -n -5 -p 6001
# sample output: renice: failed to set priority for 6001 (process ID): Permission
denied
```

Limits:

```
ulimit -n
# sample output: 1024
ulimit -Hn
# sample output: 1048576
ulimit -n 256
ulimit -n
# sample output: 256
```

Enforcement with a subshell:

```
( ulimit -t 1; while :; do :; done )
# sample output: Killed
```

The loop is stopped after about one second of CPU time (SIGXCPU or SIGKILL depending on soft and hard limits).

Inspect another process:

```
grep 'open files' /proc/$$/limits
# sample output: Max open files            1024                 1048576
files
```

Mini-lab

1. Run `ps -o pid,ni,cmd -p $$` and `nice` to see the default 0.
2. Start `nice -n 15 sleep 200 &` and verify NI=15 with `ps`.
3. Run `renice -n 19 -p PID`; then try to lower it back with `renice -n 0 -p PID` and read the error.
4. As root (or in a container) run `sudo nice -n -5 sleep 100 &` and check NI=-5.
5. Compare CPU shares: run two `nice -n 0 sh -c 'while :; do :; done' &` loops and one with `-n 19` pinned to one CPU (`taskset -c 0`); watch `top`.
6. Run `ulimit -a` and identify open files, processes and stack size.
7. In a subshell run `( ulimit -n 20; ls /proc/self/fd | wc -l )`; then cause "Too many open files" by opening more with a small script.
8. Clean up with `pkill -f 'while :'` and `jobs; kill %1`.

Decision guide / failure modes

Use `nice` for batch work (backups, compression, builds) that should yield to interactive tasks; use `renice` on something already running; use `ionice` for disk-heavy jobs; use `ulimit` for quick shell-scoped caps and systemd or cgroups for services and containers. Raise `nofile` for servers that open many sockets.

Common failures:

- Believing `nice -n 19` makes a job fast or slow in absolute terms; it matters only under contention.
- Thinking a higher number means higher priority; it is the opposite.

- `Too many open files` in a busy server: soft limit too low (often 1024); raise via limits.conf or `LimitNOFILE`.
- `ulimit` settings vanishing after logout because they were only set in a shell, not persisted.
- Trying to raise a hard limit as a normal user; lowering is allowed, raising is not.
- Confusing `-n` of `nice` (adjustment) with `ulimit -n` (open files).

Common mistakes

- Raising nofile without diagnosing fd leaks
- Assuming nice makes a slow disk fast
- Ignoring OOM killer symptoms

Exercises

Work in `exercises/13-nice-priority/`. Read that folder's `README.md` fully before coding.

```
cd exercises/13-nice-priority
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement higher_nice_means(). It takes no arguments and returns the short string "lower-priority", describing what a higher niceness value means for a process: the process is nicer to others, so the scheduler gives it a smaller share of CPU time when there is contention. Return exactly "lower-priority" in lowercase with a hyphen. Niceness ranges from –20 (highest priority) to 19 (lowest priority), the default is 0, and only root may decrease a niceness value. This is the key idea behind nice and renice.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/13-nice-priority/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Does higher nice mean more CPU?
2. Why limit open files?
3. What tech do containers use for limits?

Further reading

- https://man7.org/linux/man-pages/man1/nice.1.html